IMPERIAL

Department of Computing
70015: Mathematics for Machine Learning
Lecturer: Matthew Wicker

Lecture 05: Convexity, Convergence, & Optimization

Additional Resources

- Covering basic ideas: Numerical Computation, Chapter 4 (Page 78) Bengio et al. (2017).
- Practical but introductory discussion of grad. descent in deep models, Chapter 8 (Page 271) Bengio et al. (2017).
- More comprehensive overall coverage: Continuous Optimization, Chapter 7 (Page 225) up to 7.4 Deisenroth et al. (2020).

5.1 Learning as Optimization

In previous lectures, we have looked at gradient descent as a way to find the *best* parameter for given model and dataset. In this lecture we will inspect this process in a bit more detail. Let us begin by recalling the optimization problem we want to solve:

$$\theta^\star = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i}^{N} \mathcal{L}(f^\theta(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}))$$

In this equation, we say that $\theta^\star$ is the *optimal* parameter. Using the argmin operator returns location at which the following minimization problem attains its solution:

$$\min_{\theta} \frac{1}{N} \sum_{i}^{N} \mathcal{L}(f^\theta(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}))$$

The core algorithm that we presented for solving this optimization problem, stated in more computer science flavor here and as a mathematical process below, is gradient descent:

Algorithm 1 Gradient Descent
Input: X - Inputs, Y - Labels, α - Learning rate, K - Number of iterations

$\theta^{(1)} \leftarrow$ Random Initialization
for $i \in [K]$ **do**
  $l = \mathcal{L}(\boldsymbol{y}, f^{\theta}(\boldsymbol{X}))$
  $\theta^{(i+1)} \leftarrow \theta^{(i)} - \alpha \nabla_{\theta} l$
end for
return θ_K

5.2 Progress Bounds for Gradient Descent

One critical question we might have about gradient descent is: *does it work*? While practical experiments have established it as the *de facto* work-horse of machine learning methods, in this course we would like to use mathematical tools to understand and analyze why and when it works. To that end, we will first introduce an assumption about the gradient of our function:

Definition 5.2.1. Lipschitz Continuous Gradient A function $\mathcal{L}(\theta) : \mathbb{R}^n \to \mathbb{R}$ has a Lipschitz continuous gradient if there exists a constant $L > 0$ such that for all $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^n$, the following inequality holds:

$$\|\nabla \mathcal{L}(\boldsymbol{u}) - \nabla \mathcal{L}(\boldsymbol{v})\| \leq L\|\boldsymbol{u} - \boldsymbol{v}\|.$$

The constant L is called the *Lipschitz constant*, and it bounds the rate of change of the gradient.

For a twice-differentiable function $\mathcal{L}(\theta)$ (i.e., $\mathcal{L} \in C^2$), a variation of the multivariate Taylor expansion around the point θ_0 can be written as follows. Given two points $\boldsymbol{v}$ and $\boldsymbol{w} \in \mathbb{R}^n$, we have:

$$\mathcal{L}(\boldsymbol{w}) \approx \mathcal{L}(\boldsymbol{v}) + \nabla \mathcal{L}(\boldsymbol{v})^{\top}(\boldsymbol{w} - \boldsymbol{v}) + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{v})^{\top} H(\boldsymbol{v})(\boldsymbol{w} - \boldsymbol{v}),$$

where $H(\boldsymbol{v})$ is the Hessian matrix of $\mathcal{L}$ at $\boldsymbol{v}$. This expansion expresses the second-order approximation of $\mathcal{L}(\boldsymbol{w})$ in terms of $\boldsymbol{v}$. Now, by assuming the gradient of $\mathcal{L}$ is Lipschitz continuous with constant L, we can bound the second-order term. Specifically, from the Lipschitz continuity of the gradient, we have:

$$\mathcal{L}(\boldsymbol{w}) \leq \mathcal{L}(\boldsymbol{v}) + \nabla \mathcal{L}(\boldsymbol{v})^{\top}(\boldsymbol{w} - \boldsymbol{v}) + \frac{L}{2}\|\boldsymbol{w} - \boldsymbol{v}\|^2.$$

This inequality is known as the *descent lemma*, and it provides an upper bound on the function value in terms of the gradient and the Lipschitz constant.

Recall the update rule for gradient descent, where at iteration t, we update the parameter θ as follows:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{1}{L}\nabla \mathcal{L}(\theta^{(t)}),$$

where $\frac{1}{L}$ is chosen as the learning rate. Substituting $\theta^{(t)}$ and $\theta^{(t+1)}$ for v and w in the descent lemma, we get:

$$\mathcal{L}(\theta^{(t+1)}) \leq \mathcal{L}(\theta^{(t)}) - \frac{1}{L}\|\nabla \mathcal{L}(\theta^{(t)})\|^2 + \frac{L}{2}\left\|\theta^{(t+1)} - \theta^{(t)}\right\|^2.$$

The gradient norm that appears to replace the first order term in this expansion is due to the fact that the gradient descent rule is $\theta^{(t+1)} = \theta^{(t)} - \alpha\nabla\mathcal{L}(\theta^{(t)})$. Subtracting $\theta^{(t)}$ to the other side we have:

$$\theta^{(t+1)} - \theta^{(t)} = -\alpha\nabla\mathcal{L}(\theta^{(t)})$$

and plugging this identity into the first order term we have: $-\alpha\nabla\mathcal{L}(\theta^{(t)})^\top\nabla\mathcal{L}(\theta^{(t)})$ which by definition is the 2-norm of the gradient. To complete the transformation, we use the same gradient update rule identity to get that $\left\|\theta^{(t+1)} - \theta^{(t)}\right\| = \frac{1}{L}\|\nabla\mathcal{L}(\theta^{(t)})\|$, and substituting this in to our expansion and simplifying gives:

$$\mathcal{L}(\theta^{(t+1)}) \leq \mathcal{L}(\theta^{(t)}) - \frac{1}{2L}\|\nabla\mathcal{L}(\theta^{(t)})\|^2.$$

This shows that with each iteration of gradient descent, the function value decreases by at least $\frac{1}{2L}\|\nabla\mathcal{L}(\theta^{(t)})\|^2$, which proves that gradient descent makes progress towards minimizing $\mathcal{L}$ at each step.

5.3 Convex Optimization

We have now see that each step of gradient descent moves us in a direction that provably improves the loss of our model. But as we continue in that direction, where will we end up? We will begin to answer to this question by considering the setting of *convex* optimization. An optimization problem is convex if and only if the function we are optimizing is convex and the domain of the problem is convex. For a function to be convex it must satisfy:

Definition 5.3.1. Convex Function We say that a function $\mathcal{L}(\theta) : \mathbb{R}^n \to \mathbb{R}$ is convex if and only if:

$$\mathcal{L}(\alpha\theta + (1-\alpha)\theta') \leq \alpha\mathcal{L}(\theta) + (1-\alpha)\mathcal{L}(\theta')$$

This definition is formulated in terms of the secant of a function and essentially says that every function value between $\mathcal{L}(\theta)$ and $\mathcal{L}(\theta')$ must lie below or on the secant. We also require that the domain of the function is a convex set. A convex set is defined as:

Definition 5.3.2. Convex Set We say that a set $C \subseteq \mathbb{R}^n$ is convex if $\forall x, y \in C$ and $\forall\alpha \in [0, 1]$:

$$(\alpha x + (1-\alpha)y) \in C$$

An important practical tool for determining if a function is convex is by examining its second-order derivative, known as the Hessian. For twice-differentiable functions, convexity can be characterized by the positive semi-definiteness of the Hessian matrix.

Theorem 5.3.3. ***Hessian Test for Convexity*** *Let $\mathcal{L}(\theta) : \mathbb{R}^n \to \mathbb{R}$ be a twice-differentiable function. The function $\mathcal{L}$ is convex if and only if its Hessian matrix $H(\theta) = \nabla^2\mathcal{L}(\theta)$ is positive semidefinite (PSD) for all $\theta \in \mathbb{R}^n$. That is, for all θ and for all vectors $v \in \mathbb{R}^n$,*

$$v^\top H(\theta)v \geq 0.$$

Proof. The proof follows from the fact that for any twice-differentiable function $\mathcal{L}(\theta)$, the second-order Taylor expansion of $\mathcal{L}$ around a point θ_0 is given by:

$$\mathcal{L}(\theta) \approx \mathcal{L}(\theta_0) + \nabla\mathcal{L}(\theta_0)^\top(\theta - \theta_0) + \frac{1}{2}(\theta - \theta_0)^\top H(\theta_0)(\theta - \theta_0).$$

If the Hessian $H(\theta_0)$ is positive semidefinite, the quadratic term is non-negative, meaning that the function lies above its tangent plane at θ_0. This is a defining property of a convex function (that you are asked to show as an exercise), and thus, the function $\mathcal{L}$ is convex. □

5.3.1 Global Optimum in Convex Optimization

Given the definitions of convex functions and convex sets, we now turn to the key result: a convex function has a unique global minimum over a convex set, provided it attains a minimum. Thus, if the optimization problem posed by machine learning model/problem turn out to be convex, then we know there is only one value $\theta^\star$ can attain.

Theorem 5.3.4. ***Global Maximum of a Convex Function*** *Let $\mathcal{L}(\theta) : \mathbb{R}^n \to \mathbb{R}$ be a convex function and let $C \subseteq \mathbb{R}^n$ be a convex set. If $\mathcal{L}$ attains a local maximum at $\theta^* \in C$, then θ^* is a global maximum. Moreover, the maximum is unique if $\mathcal{L}$ is strictly convex.*

Proof. Suppose $\mathcal{L}$ is convex, and let θ^* be a local minimum of $\mathcal{L}$ in C. Then, for some neighborhood $N \subseteq C$ around θ^*, we have $\mathcal{L}(\theta) \geq \mathcal{L}(\theta^*)$ for all $\theta \in N$. Suppose, for the sake of contradiction, that there exists some $\tilde{\theta} \in C$ such that $\mathcal{L}(\tilde{\theta}) < \mathcal{L}(\theta^*)$. Consider the line segment $S(\alpha) = \alpha\theta^* + (1-\alpha)\tilde{\theta}$ for $\alpha \in [0, 1]$, noting that $S(\alpha) \in C$ by the convexity of C. Then, by the convexity of $\mathcal{L}$, we have:

$$\mathcal{L}(S(\alpha)) \leq \alpha\mathcal{L}(\theta^*) + (1-\alpha)\mathcal{L}(\tilde{\theta}) < \alpha\mathcal{L}(\theta^*) + (1-\alpha)\mathcal{L}(\theta^*) = \mathcal{L}(\theta^*) \quad \text{for all} \quad \alpha \in (0, 1).$$

We can choose α sufficiently close to 1 such that $S(\alpha) \in N$; then, by the definition of N, we have $\mathcal{L}(S(\alpha)) \geq \mathcal{L}(\theta^*)$. However, from the inequality above, we also have $\mathcal{L}(S(\alpha)) < \mathcal{L}(\theta^*)$, a contradiction. It follows that $\mathcal{L}(\theta^*) \leq \mathcal{L}(\theta)$ for all $\theta \in C$, so θ^* is a global minimum of $\mathcal{L}$ in C.

For uniqueness, suppose θ_1^* and θ_2^* are two distinct global maxima. If $\mathcal{L}$ is strictly convex, we have for any $\alpha \in (0, 1)$,

$$\mathcal{L}(\alpha\theta_1^* + (1-\alpha)\theta_2^*) < \alpha\mathcal{L}(\theta_1^*) + (1-\alpha)\mathcal{L}(\theta_2^*) = \mathcal{L}(\theta_1^*) = \mathcal{L}(\theta_2^*),$$

which contradicts the assumption that θ_1^* and θ_2^* are global maxima. Therefore, the maximum must be unique. □

5.4 Convergence in Machine Learning

We have now seen that, at least for convex functions, one step of gradient descent improves our loss, and we have shown that for convex optimization problems there is unique, global minimum.

What we have not yet shown is if, after many steps of gradient descent, we reach this single global optimum and if we do (as we might think) how quickly? And under what conditions? We call the answer to these questions convergence and the rate of convergence, respectively.

There are many formal definitions of convergence (e.g., convergence in probability, convergence to distribution) each with their own specific criteria and use cases. The most widely known form of convergence, that most students study around their second year of calculus is something like the following:

Definition 5.4.1. Convergence A series $x_1, x_2, \ldots, x_n$ is said to *converge* to a limit T if for any $\epsilon > 0$ we have an integer K such that $\forall M > K, |x_M - T| < \epsilon$.

In calculus, this definition is typically investigated in the setting of arithmetic or geometric series. But how can the mathematical idea of convergence be employed as a tool for analysis or reasoning about our models? One may notice, that in our algorithm we defined a series of parameters as the result of our gradient descent equations: $\theta^{(1)}, \theta^{(2)}, \ldots, \theta^{(K)}$. Given this series we can then answer the question we have asked and pose further questions similar to those we might ask in calculus, namely:

- Under what conditions does this series converge?
- What does the series converge to?
- What do we want the series converge to?

The last question we identified at the beginning of lecture. In particular, we would like for our parameter to converge to:

$$\theta^\star := \operatorname*{argmin}_\theta \frac{1}{N} \sum_i^N \mathcal{L}(f^\theta(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}))$$

That is, we would like that for some finite K that $|\theta_K - \theta^\star| < \epsilon$. Before tackling this when/if this can happen in general terms, lets analyze the sequence of parameter in the case when we know the closed form for the value $\theta^\star$.

5.5 Convergence in Linear Models

We will prove convergence in the setting of our linear regression model. The reason this is a strong practical choice is that we know what we would like to converge to! Thus, we return to our linear regression model in the supervised learning setting. Recall that we assume we have input features $\boldsymbol{x} \in \mathbb{R}^{D \times 1}$ and labels $y \in \mathbb{R}$. Recall the model is:

$$f(\boldsymbol{x}, \theta) = \boldsymbol{x}^\top \theta, \quad \hat{y} = f(\boldsymbol{x}, \theta)$$

And we can express the loss with:

$$\mathcal{L}(\theta) = \frac{1}{2}\sum_{i=1}^{N}\big(y^{(i)} - f(\boldsymbol{x}^{(i)}, \theta)\big)^2$$

We can rewrite our loss in matrix form:

$$\mathcal{L}(\theta) = \frac{1}{2}||\boldsymbol{y} - \boldsymbol{X}\theta||_2^2$$

Now that we have a particular form for our loss, we can recall that if our gradient descent algorithm converges we would like for it to converge to the value $(\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}$. In addition, given that we have the form of the loss we would like to minimize (the MSE of the linear model), we can be more specific in our analysis of Algorithm 1 by plugging in the closed form of the gradient. We now isolate the update equation in our algorithm (line 4) and expand out the gradient of the loss term:

$$\begin{aligned}\theta^{(i+1)} &= \theta^{(i)} - \alpha\nabla_{\theta^{(i)}}\mathcal{L}(\mathbf{y}, f^{\theta^{(i)}}(\mathbf{X})) \\ &= \theta^{(i)} - \alpha\nabla_{\theta^{(i)}}\frac{1}{2}||\boldsymbol{y} - \boldsymbol{X}\theta_i||_2^2 \\ &= \theta^{(i)} - \alpha\boldsymbol{X}^\top(\boldsymbol{X}\theta^{(i)} - \boldsymbol{y})\end{aligned}$$

This formula is almost what we want, but not quite. As in our previous calculus education, we would like to come up with a recurrence relation that is in terms of our term of interest, θ. With a bit of algebra we can isolate:

$$= (\mathbf{I} - \alpha\boldsymbol{X}^\top\boldsymbol{X})\theta^{(i)} + \alpha\boldsymbol{X}^\top\boldsymbol{y}$$

This is a key step in our analysis because it allows us to exactly identify what kind of sequence we are looking at, in this case an arithmetico-geometric sequence: a sequence made of of repeated multiplications and additions. To begin deciding what to do, we want to look at abstractly at what form our update takes. In this case, the sequence is a matrix multiplication and a vector addition so we could abstract our update to:

$$\theta^{(t+1)} = \boldsymbol{B}\theta^{(t)} + \mathbf{c}, \quad t \geq 0,$$

While this is a great observation to make, the next step in our analysis would be to understand if there is a clean way to reason about what happens when we apply our update multiple times. In this case, our sequence would expand as:

$$\boldsymbol{B}\big(\boldsymbol{B}(\boldsymbol{B}\theta^{(1)} + \mathbf{c}) + \mathbf{c}\big) + \mathbf{c}$$

While our previous calculus/discrete math education may have nice results for linear recurrence relations the above relation is not strictly linear, due to the addition of $\mathbf{c}$. Instead, it is an *affine* relation. We can immediately see that such a relation would be much nicer to analyze if we could

get the sequence a form such that the additive structure imposed by $+\mathbf{c}$ would cancel out. This would be the case if we could express our sequence in the following form:

$$\theta^{(t+1)} = \boldsymbol{A}(\theta^{(t)} + \beta) - \beta, \quad \text{for some } \boldsymbol{A}, \beta.$$

The key idea here is that all of the β terms cancel when we expand out and so we are left with simply t matrix multiplications and a single addition and subtraction after t iterations of our update. Thus, we would have the form:

$$\theta^{(t)} = \boldsymbol{A}^t(\theta^{(1)} + \beta) - \beta$$

after t gradient descent updates. That is, expressing the sequence in the above form allow us to neatly express the sequence in terms of a simple matrix power operation. Readers can expand out the newly suggested form to check for themselves that this is the case. Of course, we want our update in this form, but we will need to do some work to get there. In particular, we must solve for $\boldsymbol{A}$ and β:

$$\begin{aligned}
\theta^{(t+1)} =& \mathbf{A}(\theta^{(t)} + \boldsymbol{\beta}) - \boldsymbol{\beta} = \boldsymbol{B}\theta^{(t)} + \boldsymbol{c} \\
\Leftrightarrow \quad & \mathbf{A}\theta^{(t)} + (\mathbf{A} - \mathbf{I})\boldsymbol{\beta} = \boldsymbol{B}\theta^{(t)} + \boldsymbol{c} \\
\Leftrightarrow \quad & \boldsymbol{A} = \boldsymbol{B}, \quad \boldsymbol{\beta} = (\boldsymbol{B} - \mathbf{I})^{-1}\boldsymbol{c}
\end{aligned}$$

Now that we have solved for $\boldsymbol{A}$ and β in terms of B and c (recall that this is just changing the sequence from one form to another), we can simply plug our values of $\boldsymbol{A}$ and β into get:

$$\Rightarrow \quad \theta^{(t+1)} = \boldsymbol{B}(\theta^{(t)} + (\boldsymbol{B} - \mathbf{I})^{-1}\boldsymbol{c}) - (\boldsymbol{B} - \mathbf{I})^{-1}\boldsymbol{c}$$

$$\Rightarrow \quad \theta^{(t)} = \boldsymbol{B}^t(\theta^{(1)} + (\boldsymbol{B} - \mathbf{I})^{-1}\boldsymbol{c}) - (\boldsymbol{B} - \mathbf{I})^{-1}\boldsymbol{c}$$

Great, now let us recall what the form of $\boldsymbol{B}$ and $\boldsymbol{c}$ were from our red equation (our gradient descent update rule for linear regression), specifically:

$$\theta^{(t+1)} = (\mathbf{I} - \alpha\boldsymbol{X}^\top\boldsymbol{X})\theta^{(t)} + \alpha\boldsymbol{X}^\top\boldsymbol{y}$$

So we have that:

$$\begin{aligned}
\boldsymbol{A} &= (\mathbf{I} - \alpha\mathbf{X}^\top\mathbf{X}) \\
\beta &= ((\mathbf{I} - \alpha\boldsymbol{X}^\top\boldsymbol{X}) - \mathbf{I})^{-1}\alpha\boldsymbol{X}^\top\boldsymbol{y}
\end{aligned}$$

Of course this is just the result of plugging in with no simplification, so we ought to simplify β (we include the algebra for completeness):

$$\begin{aligned}
((\mathbf{I} - \alpha\mathbf{X}^\top\mathbf{X}) - \mathbf{I})^{-1}\alpha\mathbf{X}^\top\mathbf{y} &= (-\alpha\mathbf{X}^\top\mathbf{X})^{-1}\alpha\mathbf{X}^\top\mathbf{y} \\
&= -(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}
\end{aligned}$$

This quantity is familiar to us. In Lecture 2, we derived this as the least squares estimate for our linear model. By calling this value θ^*, we have arrived at the following expression for our update:

$$\theta^{(t)} = (\mathbf{I} - \alpha\boldsymbol{X}^\top\boldsymbol{X})^t(\theta^{(1)} - \theta^*) + \theta^*, \quad \theta^* = (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}$$

Thus, we can observe that our sequence will converge to the desired value if $(\mathbf{I} - \alpha \boldsymbol{X}^\top \boldsymbol{X})^t(\theta^{(1)} - \theta^*) \to \mathbf{0}$. But does this occur? In order to find out lets write out the formula for the distance between the result of our algorithm and the desired minimizer, that is he ℓ_2 distance between $\theta^{(t)}$ and θ^*:

$$\begin{aligned} ||\theta^{(t)} - \theta^*||_2^2 &= ||(\mathbf{I} - \alpha \boldsymbol{X}^\top \boldsymbol{X})^t(\theta^{(1)} - \theta^*)||_2^2 \\ &= |(\theta^{(1)} - \theta^*)^\top (\mathbf{I} - \alpha \boldsymbol{X}^\top \boldsymbol{X})^{2t}(\theta^{(1)} - \theta^*)| \end{aligned}$$

Recall from our review of linear algebra that we can use eigen decomposition in order to bound the above product. The relevant fact is: $\lambda_{min}(\boldsymbol{A})||\boldsymbol{x}||_2^2 \leq \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \leq \lambda_{max}(\boldsymbol{A})||\boldsymbol{x}||_2^2$. When substituting for the relevant terms we get:

$$||\theta^{(t)} - \theta^*||_2^2 \geq \lambda_{min}((\mathbf{I} - \alpha \boldsymbol{X}^\top \boldsymbol{X})^{2t})||\theta^{(1)} - \theta^*||_2^2$$
$$||\theta^{(t)} - \theta^*||_2^2 \leq \lambda_{max}((\mathbf{I} - \alpha \boldsymbol{X}^\top \boldsymbol{X})^{2t})||\theta^{(1)} - \theta^*||_2^2$$

Convergence properties in difference cases:

1. $\lambda_{max} < 1$: always converge
2. $\lambda_{min} \geq 1$: always diverge
3. $\lambda_{min} < 1$ but $\lambda_{max} \geq 1$: convergence depending on $\theta^{(1)}$

While this gives us a guide for when our algorithm will converge, we would ideally like to try to control the maximum eigenvalue such that we guarantee convergence. So, lets use our knowledge of eigenvalues to figure out how we can manipulate α in order to ensure convergence. Lets start by assuming that we have a value λ that is an eigenvalue of $\boldsymbol{X}^\top \boldsymbol{X}$. Lets review some simple facts about eigenvalues by thinking about an eigenvalue λ of a matrix A. We know that λ^2 is an eigenvalue of A^2, we know that $c\lambda$ is an eigenvalue of cA and we know that $1 - \lambda$ is an eigenvalue of $I - A$. Putting all of these together we get that if λ is an eigenvalue of $\boldsymbol{X}^\top \boldsymbol{X}$, then we have that $\left(1 - \alpha\lambda\right)^2$ is an eigenvalue of $\left(\mathbf{I} - \alpha \boldsymbol{X}^\top \boldsymbol{X}\right)^2$. Now that we have the form of an eigenvalue of our matrix of interest, we need to isolate α. Recalling that $\boldsymbol{X}^\top \boldsymbol{X}$ is PSD, we have that:

$$\alpha < \frac{2}{\lambda_{max}(\boldsymbol{X}^\top \boldsymbol{X})}$$

Learning Objectives and Techniques

- Convexity (tangent, secant, and Hessian definitions)
- Definition of convergence
- Proving convergence in arithmetic-geometric series
- Use of eigenvalues and eigenvectors (in reference to the above)

Bibliography

Bengio, Y., Goodfellow, I., and Courville, A. (2017). *Deep learning*, volume 1. MIT press Cambridge, MA, USA.

Deisenroth, M. P., Faisal, A. A., and Ong, C. S. (2020). *Mathematics for machine learning*. Cambridge University Press.